NOMBRE Y APELLIDOS:
FECHA: 18/03/2026
EXP.: EXAMEN

Python y Django (IFCT095PO)

Duración: 60 minutos | Puntuación total: 100 puntos
Puntuación mínima:

- 15/40 en test
- 15/60 en práctico

Instrucciones:

Clonad el repositorio:

```
git clone https://github.com/elMestreAcademy/2026-IFCT095PO-GR1-prueba
```

Responded sobre el fichero `RESPUESTA.md`

Test (10 preguntas, 1 punto c/u)

1. ¿Dónde se accede a los datos validados de un formulario Django?

- a) `form.data`
- b) `form.cleaned_data`
- c) `form.validated_data`
- d) `form.fields`

2. ¿Cuál de estas es una estructura de datos inmutable en Python?

- a) `list`
- b) `dict`
- c) `set`
- d) `tuple`

3. ¿Cuál es el resultado de ejecutar `type([1, 2, 3])` en Python?

- a) `<class 'tuple'>`
- b) `<class 'list'>`
- c) `<class 'array'>`
- d) `<class 'set'>`

4. ¿Qué valor de `on_delete` elimina en cascada los objetos dependientes?

- a) `models.CASCADE`
- b) `models.DELETE`
- c) `models.REMOVE_ALL`
- d) `models.DESTROY`

5. ¿Qué token debe incluirse en un formulario POST en Django para prevenir ataques CSRF?

- a) `{% csrf_field %}`
- b) `{% csrf_token %}`
- c) `{{ csrf_hash }}`
- d) `{% security_token %}`

6. ¿Qué método de un formulario Django comprueba que los datos enviados son válidos?

- a) `form.check()`
- b) `form.validate()`
- c) `form.is_valid()`
- d) `form.clean_all()`

7. ¿Qué comando crea un superusuario para el panel de administración Django?

- a) `python manage.py createadmin`
- b) `python manage.py createsuperuser`
- c) `python manage.py addadmin`
- d) `python manage.py superuser`

8. ¿Qué fichero contiene la configuración principal de un proyecto Django?

- a) `urls.py`
- b) `settings.py`
- c) `wsgi.py`
- d) `models.py`

9. ¿Qué backend de sesión almacena los datos en la base de datos por defecto?

- a) `django.contrib.sessions.backends.cache`
- b) `django.contrib.sessions.backends.db`
- c) `django.contrib.sessions.backends.file`
- d) `django.contrib.sessions.backends.signed_cookies`

10. ¿Cuál es la salida de `typeof null` en JavaScript?

- a) `"null"`
- b) `"undefined"`
- c) `"object`
- d) `"boolean"`

Pregunta	1	2	3	4	5	6	7	8	9	10
Respuesta										

Supuesto práctico

Los tres ejercicios trabajan sobre el proyecto Django prueba/, con la app biblioteca y sus modelos Autor y Libro. Escribid el código solicitado. Se valorará corrección sintáctica, funcionalidad y buenas prácticas. Podéis consultar el código existente del proyecto como referencia.

Modelos existentes en `biblioteca/models.py`:

Ejercicio 1: Nuevo modelo: Reseña (20 puntos)

Se quiere añadir a la app `biblioteca` la posibilidad de que los usuarios dejen reseñas sobre los libros. Defina en `models.py` un nuevo modelo llamado `Resena` con los siguientes campos:

- `libro`: clave foránea al modelo `Libro` (si se borra el libro, se borran sus reseñas)
- `autor_nombre`: texto, máximo 100 caracteres, obligatorio (nombre de quien escribe la reseña)
- `puntuacion`: entero (representará un valor de 1 a 5)
- `comentario`: texto largo, puede estar vacío
- `fecha`: fecha/hora, se asigna automáticamente al crear el registro

Incluya:

- Un método `__str__` que devuelva un texto con el formato: "Reseña de {autor_nombre} sobre {libro.titulo}"
- Una clase `Meta` que ordene por fecha descendente (las más recientes primero)

Finalmente, registre el modelo `Resena` en `admin.py` para que sea gestionable desde el panel de administración.

Ejercicio 2: Vista + Template: Reseñas de un libro (20 puntos)

Escriba:

a) Una nueva entrada en `urls.py` de la app `biblioteca` para la ruta `libro/<int:libro_id>/resenas/` con el nombre `resenas-libro`**. (3 puntos)**

b) Una vista basada en función llamada `resenas_libro` en `views.py` que: **(10 puntos)**

- Reciba `request` y `libro_id`
- Obtenga el libro correspondiente o devuelva un 404 si no existe (use `get_object_or_404`)
- Obtenga todas las reseñas de ese libro (ya ordenadas por el `Meta` del modelo)
- Calcule la puntuación media de las reseñas (use `aggregate` con `Avg`). Si no hay reseñas, la media será `None`
- Pase al template `biblioteca/resenas.html` el libro, las reseñas y la media

c) El template `biblioteca/resenas.html` que: **(7 puntos)**

- Extienda de `base.html`
- Muestre un `<h1>` con el texto "Reseñas de {titulo del libro}"
- Si existe puntuación media, muestre un párrafo con: "Puntuación media: X.X / 5"
- Recorra las reseñas mostrando en cada una: autor_nombre, puntuación y comentario
- Si no hay reseñas, muestre: "Este libro aún no tiene reseñas."
- Incluya un enlace de vuelta a la lista de libros usando `{% url 'biblioteca:lista-libros' %}`

Ejercicio 3: Formulario con validación: Crear reseña (20 puntos)

Cree un `ModelForm` llamado `ResenaForm` en `forms.py` que:

- Trabaje con el modelo `Resena`
- Incluya solo los campos: `autor_nombre`, `puntuacion`, `comentario`
- Personalice el widget de `autor_nombre` para usar `attrs={'class': 'form-control', 'placeholder': 'Tu nombre'}`
- Personalice el widget de `comentario` para usar `forms.Textarea` con `attrs={'class': 'form-control', 'rows': 4}`
- Implemente una validación personalizada `clean_puntuacion` que compruebe que el valor está entre 1 y 5 (ambos incluidos). Si no lo está, lance un `ValidationError` con el mensaje: `"La puntuación debe estar entre 1 y 5."`
- Implemente una validación personalizada `clean_autor_nombre` que compruebe que el nombre tiene al menos 2 caracteres. Si no, lance un `ValidationError` con el mensaje: `"El nombre debe tener al menos 2 caracteres."`